

SCD

(SLOWLY CHANGING DIMENSION) TYPES

A Complete Guide with SQL Examples

Master the most common Slowly Changing Dimensions (SCD) types and learn how to implement them!



Abhishek Agrawal
Data Engineer



SCD Type 1 – Overwrite (No History Tracking)

Definition: In **SCD Type 1**, old values are **overwritten** with new values, and no history is maintained.

Use Case: Updating a customer's email address.

SQL Code Example:

```
MERGE INTO customers AS target
USING updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
  UPDATE SET target.email = source.email;
```

Key Point: Use **Type 1** when historical changes are **not important**.

SCD Type 2 – History Tracking

Definition: In **SCD Type 2**, a new row is added for each change, maintaining the complete history.

Use Case: Tracking customer address changes over time.

SQL Code Example:

```
MERGE INTO customers_history AS target
USING updates AS source
ON target.customer_id = source.customer_id AND target.end_date IS NULL

-- If a matching record is found and attributes have changed, end the old record
WHEN MATCHED AND (target.name <> source.name OR target.address <> source.address)
THEN
    UPDATE SET target.end_date = GETDATE()

-- Insert a new record after ending the previous one
WHEN MATCHED AND (target.name <> source.name OR target.address <> source.address)
THEN
    INSERT (customer_id, name, address, start_date, end_date)
    VALUES (source.customer_id, source.name, source.address, GETDATE(), NULL)

-- If no matching record exists in the target table, insert the new record
WHEN NOT MATCHED BY TARGET THEN
    INSERT (customer_id, name, address, start_date, end_date)
    VALUES (source.customer_id, source.name, source.address, GETDATE(), NULL);
```

Key Point: Use **Type 2** to track **historical changes** for auditing or reporting purposes.



SCD Type 3 – Limited History

Definition: In **SCD Type 3**, only the previous value is kept in an additional column.

Use Case: Tracking a customer's previous job title.

SQL Code Example:

```
MERGE INTO employees AS target
USING updates AS source
ON target.employee_id = source.employee_id
WHEN MATCHED THEN
    UPDATE SET target.previous_job_title = target.current_job_title,
               target.current_job_title = source.job_title
WHEN NOT MATCHED THEN
    INSERT (employee_id, name, current_job_title, previous_job_title)
    VALUES (source.employee_id, source.name, source.job_title, NULL);
```

Key Point: Use **Type 3** when only **recent history** matters.



SCD Type 4 – Separate History Table

Definition: In **SCD Type 4**, historical data is stored in a **separate table**.

Use Case: Maintaining a customer's address history in a dedicated table.

SQL Code Example:

```
-- Insert current data into the main table
MERGE INTO customers AS target
USING updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
    UPDATE SET target.address = source.address;

-- Insert historical data into the history table
INSERT INTO customers_history (customer_id, name, address, change_date)
SELECT customer_id, name, address, current_date()
FROM updates;
```

Key Point: Use **Type 4** when you want to **separate historical data** for performance optimization.



SCD Type 6 – Hybrid (1+2+3)

Definition: **Type 6** is a **hybrid approach** that combines SCD **Types 1, 2, and 3** to track both **current and historical** values.

Use Case: Tracking both the **current address and the previous address** of a customer while keeping a full history.

SQL Code Example:

```
MERGE INTO customers AS target
USING updates AS source
ON target.customer_id = source.customer_id
WHEN MATCHED THEN
    UPDATE SET target.previous_address = target.current_address,
               target.current_address = source.address,
               target.end_date = current_date()
WHEN NOT MATCHED THEN
    INSERT (customer_id, name, current_address, previous_address, start_date, end_date)
    VALUES (source.customer_id, source.name, source.address, NULL, current_date(), NULL);
```

Key Point: Use **Type 6** when you need to track both **current and historical changes** in a single table.



**Follow for more
content like this**



Abhishek Agrawal

Azure Data Engineer

